

准考证号: EE2003

考试形式: 闭卷

姓名: _____ 学号: _____ 班级: _____

题号	一	二	三	四()	四()	四()	四()	总分
得分								

*第四题中选做的题号请填入上表

第二题(选择题)答题卡

6	7	8	9	10	11	12	13	14	15
16	17	18	19	20	21	22	23	24	25
26	27	28	29						

第三题(判断题)答题卡

30	31	32	33	34	35	36	37	38	39
40	41	42	43						

一、 填空题 (10 分, 请直接在试卷上答题)

- 1、 当代的 CPU 芯片, 往往在单颗处理器芯片内集成冯·诺依曼结构计算机五个基本部件中的_____和_____。
- 2、 使用流水线后, 指令执行过程会出现不同类型的相关冲突, 其中因程序中分支跳转引起的冲突被称为_____。此类冲突产生时, “排空” 流水线所带来的延迟被称为_____。
- 3、 I/O 接口的典型电路中, 常包括输入数据(缓存)寄存器、输出数据(缓存)寄存器、_____、_____等 I/O 端口寄存器。
- 4、 Cortex-M3 处理器进入异常处理子程序时, 被中断程序中下一条待执行指令的地址常被称为_____, 而被中断程序的运行环境(如 CPU 内部各类寄存器的内容)常被称为_____。
- 5、 实验过程中, IDE 采用_____软件(可进行交叉编译), 宿主机通过_____仿真器(Debugger)连接实验箱并烧写代码。



二、单项选择题 (36 分, 每小题 1.5 分, 请试卷首页答题卡上答题)

- 6、 以下关于 2023H、0010 1111 0010 0001B、2049D 三个数描述正确的有 ()
- I. 若三个数均为无符号数, 则最大的数是 2023H
 - II. 若三个数均为无符号数, 则最大的数是 0010 1111 0010 0001B
 - III. 若三个数均为无符号数, 则最大的数是 2049D
 - IV. 若三个数均为无符号数, 则最小的数是 2023H
 - V. 若三个数均为无符号数, 则最小的数是 0010 1111 0010 0001B
 - VI. 若三个数均为无符号数, 则最小的数是 2049D
- A) I、V C) III、IV E) IV、V
B) II、V D) III、V F) ABCDE 均不正确
- 7、 某位长为 16 的计算机中, 带符号整数用补码表示。变量 a 、 b 、 c 、 d 的机器数分别为 7FFE_H、0002_H、FFFE_H、8001_H, 下列结论中, 正确的是 ()
- I. $a+b$ 运算不会出现溢出
 - II. $a+c$ 运算不会出现溢出
 - III. $b+c$ 运算不会出现溢出
 - IV. $b+d$ 运算不会出现溢出
 - V. $c+d$ 运算不会出现溢出
- A) I、II、V C) II、III、IV E) III、IV、V
B) I、III、V D) II、III、V F) ABCDE 均不正确
- 8、 已知单精度 float 型数据用 IEEE754 标准 (格式参见附录 5) 表示。假定变量 x 的类型是 float, x 保存在寄存器中, 寄存器值为 F000 0000_H 时, x 的值是 ()
- A) -2^{27} C) -2^{96} E) -2^{97}
B) 2^{27} D) 2^{96} F) 2^{97}
- 9、 以下关于微程序控制器中机器指令、微指令、微操作描述正确的是 ()
- I. 每一条机器指令由一段用微指令编成的微程序来解释执行
 - II. 每一条机器指令由一条微指令来执行
 - III. 每一条机器指令的执行可分解为一系列的微操作
 - IV. 每一条微指令由一条机器指令来执行
 - V. 一段机器指令组成的程序可由一条微指令来执行
 - VI. 每一条微指令由若干条机器指令组成
- A) I、II C) II、III E) IV、VI
B) I、III D) IV、V F) ABCDE 均不正确
- 10、 以下关于计算机体系结构说法正确的有 ()
- I. 流水线机制能够有效提高 CPI (Cycles Per Instruction)
 - II. 流水线机制能够有效提高 IPC (Instructions Per Cycle)
 - III. Load/Store 体系更有利于实现流水线
 - IV. RISC 相比 CISC 更有利于实现流水线
 - V. 标量处理器可以使用多条流水线来提升性能
 - VI. 超标量处理器可以使用多条流水线来提升性能
- A) I、II、III C) III、IV、V E) III、V、VI
B) II、III、IV D) IV、V、VI F) ABCDE 均不正确





- 11、若将流水线分成 Fetch、Decode、Read、Execute、WriteBack 五个阶段，以下描述正确的有（ ）
- I. 不同指令的不同阶段可以重叠执行
 - II. 各个阶段都要共用相同的功能部件
 - III. 各个阶段的执行时间最好不相等
 - IV. 每个阶段的执行时间应取 5 个阶段执行时间的最小值
 - V. 每个阶段的执行时间应取 5 个阶段执行时间的最大值
 - VI. 每个阶段的执行时间应取 5 个阶段执行时间的平均值
- A) I、III C) I、VI E) IV、VI
B) I、IV D) III、IV F) ABCDE 均不正确
- 12、以下指令序列（指令功能参考附录 4），会导致流水线数据相关的情形包括（ ）
- I. 第 1 条指令为“ADD R0, R1, R2”，第 2 条指令为“ADD R3, R4, R5”
 - II. 第 1 条指令为“ADD R0, R1, R2”，第 2 条指令为“ADD R2, R1, R0”
 - III. 第 1 条指令为“LDR R1, [R0]”，第 2 条指令为“ADD R0, R1, R2”
 - IV. 第 1 条指令为“LDR R0, [R1]”，第 2 条指令为“ADD R3, R4, R5”
 - V. 第 1 条指令为“ADD R0, R1, R2”，第 2 条指令为“LDR R1, [R0]”
 - VI. 第 1 条指令为“ADD R3, R4, R5”，第 2 条指令为“LDR R1, [R0]”
- A) I、II、III C) II、III、V E) II、IV、VI
B) II、III、IV D) III、V、VI F) ABCDE 均不正确
- 13、在多级存储体系中，缓存——主存结构的作用是解决（ ）问题。
- A) 主存与 CPU 速度不匹配
B) 主存和辅存速度不匹配
C) 辅存与 CPU 速度不匹配
D) 外设与 CPU 速度不匹配
E) 主存容量不足
F) 辅存容量不足
- 14、以下关于缓存（Cache）地址映射方式说法正确的是（ ）
- I. 全相联映射方式的缓存命中率一般会高于直接相联映射方式
 - II. 直接相联映射方式的缓存命中率一般会高于组相联映射方式
 - III. 直接相联映射方式的硬件电路比全相联映射方式要简单
 - IV. 组相联映射方式中，Cache 组内的块数一般取值较小（如 16）
 - V. 组相联映射方式中，Cache 的组数目一般取值较小（如 16）
 - VI. 实验所用 STM32F407 系列处理器采用了组相联映射方式
 - VII. PC 上所用处理器一般会采用组相联映射方式
- A) I、II C) IV、V E) VI、VII
B) III、IV D) V、VI F) ABCDE 均不正确
- 15、以下关于半导体存储器说法正确的是（ ）
- I. 半导体存储与磁存储、光存储相比，具有体积小、重量轻的优点
 - II. 半导体存储被广泛用于芯片内存储、计算机主存和外存
 - III. U 盘使用了半导体存储技术
 - IV. 硬磁盘（HDD）使用了半导体存储技术
 - V. 现代计算机上的内存条使用了半导体存储技术
 - VI. 实验所用 STM32F407 系列芯片使用了半导体存储技术



- A) I、II、III C) II、III、IV E) IV、V、VI
B) I、III、IV D) III、IV、V F) ABCDE 均不正确
- 16、以下关于计算机辅存接口说法正确的是（ ）
- I. 早期 PC 往往采用 IDE（Integrated Drive Electronics）和 ATA（Advanced Technology Attachment）连接硬磁盘（HDD）
 - II. 当代 PC 往往采用 SATA 连接硬磁盘（HDD）
 - III. 当代 PC 可以采用 SATA 连接固态硬盘（SSD）
 - IV. 当代 MCU 系统往往采用 SATA 连接硬磁盘（HDD）
 - V. 当代 MCU 系统可以采用 SPI 连接外部存储芯片
 - VI. 当代 MCU 系统可以采用 I²C 连接外部存储芯片
- A) I、II、III C) III、IV、V E) IV、V、VI
B) I、IV、VI D) III、IV、VI F) ABCDE 均不正确
- 17、以下关于计算机总线说法正确的是（ ）
- I. 总线的设计思想是把不同部件均接到一个公共的通道上
 - II. 总线上的设备一般分为总线主设备和总线从设备
 - III. 连接在同一套总线上的所有设备必须按照统一的时钟工作
 - IV. 多主设备环境才需要总线仲裁
 - V. 单主设备环境也需要总线仲裁
 - VI. 单主设备的总线系统中，总线操作周期可仅包含寻址和数据传输 2 个阶段
- A) I、II、III C) II、III、IV E) IV、V、VI
B) I、II、IV D) III、IV、V F) ABCDE 均不正确
- 18、以下关于“地址”描述不正确的是（ ）
- I. 地址总线不同信号线对应到地址的不同位
 - II. 一般来说，计算机存储器地址按照字节编码
 - III. 地址可以用来寻址存储器内的存储单元
 - IV. 地址不能用来寻址 I/O 端口寄存器
 - V. 缓存（Cache）地址空间和主存地址空间是相同的地址空间
 - VI. 虚存空间中的虚拟地址可以转换为物理地址
- A) I、II C) III、IV E) V、VI
B) II、III D) IV、V F) ABCDE 均不正确
- 19、以下关于 AHB 总线描述不正确的有（ ）
- I. AHB 总线支持多主设备
 - II. AHB 总线的地址总线 and 数据总线是复用的
 - III. AHB 的读数据和写数据复用同一组信号线
 - IV. AHB 总线采用了异步总线时序
 - V. AHB 中，“驱动地址”和“驱动数据”可构成两级“流水线”操作
 - VI. AHB 总线的读写通道是分离的
- A) I、II C) II、III E) IV、VI
B) I、III D) III、V F) ABCDE 均不正确
- 20、以下关于 AHB 突发传输描述不正确的有（ ）
- I. 突发传输可以通过 HSIZE[2:0]指定传输数据块的长度
 - II. 突发传输通过 HTRANS[1:0]指示传输状态如“SEQ”、“IDLE”等



- III. 存储空间片外外设区域中有 1MB 位段区被映射为 32MB 位带别名区域
 - IV. 存储空间 SRAM 区域中有 1MB 位段区被映射为 32MB 位段带名区域
 - V. 存储空间外部 RAM 区域中有 1MB 位段区被映射为 32MB 位段带名区域
 - VI. 位带操作可以避免“读→修改→写”问题
- A) I、II C) III、IV E) V、VI
B) II、III D) IV、VI F) ABCDE 均不正确

- 26、以下关于 Cortex-M3/M4 堆栈说法不正确的是 ()
- I. 具有双栈的结构, 有 MSP 和 PSP 两个堆栈指针
 - II. MSP 和 PSP 的值在初始化程序中进行设置
 - III. 非特权线程模式只能使用 PSP 而不能使用 MSP
 - IV. 往栈内放置一个新的数据后, 栈指针数值会增加
 - V. 栈空间只能放置在 4GB 空间的 SRAM 区
 - VI. 栈空间可以放置在 4GB 空间的任何一个区
- A) I、II、III C) III、IV、V E) I、V、VI
B) II、III、IV D) IV、V、VI F) ABCDE 均不正确

- 27、以下关于 STM32F103 系列处理器时钟描述不正确的有 ()
- I. LSE 和 LSI 时钟源是不能与 PLL 配合工作的
 - II. HSE 和 HSI 时钟源由于时钟频率高, 需要从芯片外部输入
 - III. SYSCLK 既可以由芯片外部时钟源产生, 也可以由芯片内部时钟源产生
 - IV. 各个定时器的时钟均可由系统时钟 SYSCLK 分频得到
 - V. 独立看门狗模块的时钟可由系统时钟 SYSCLK 分频得到
 - VI. 各个 GPIO 端口的时钟可由系统时钟 SYSCLK 分频得到
- A) I、II C) II、V E) V、VI
B) II、III D) IV、V F) ABCDE 均不正确

- 28、以下关于“地址”描述正确的有 ()
- I. “I/O 端口地址”既可以和存储器地址统一规划, 也可以独立规划
 - II. STM32 处理器中特定的“I/O 端口地址”代表了特定的寄存器
 - III. STM32 处理器中 USART1 模块 SR 寄存器的地址不属于“I/O 端口地址”
 - IV. STM32 处理器中 GPIOA 模块 ODR 寄存器的地址属于“I/O 端口地址”
 - V. “I²C 总线规范中定义的地址”是 I²C 器件地址
 - VI. “I²C 总线规范中定义的地址”是 I²C 器件中 I/O 端口的地址
- A) I、II、III C) II、IV、VI E) IV、V、VI
B) II、III、IV D) III、IV、V F) ABCDE 均不正确

- 29、以下关于 STM32F103 单片机 USART 接口说法正确的有 ()
- I. USART 模块既支持异步串行通信, 也支持同步串行通信
 - II. USART 模块可以触发中断
 - III. USART 中断的优先级是固定的
 - IV. USART 模块使用前不需要配置引脚复用方式
 - V. USART 模块可通过对数据寄存器 DR 的读/写实现收/发
 - VI. 所有 USART 模块均挂载在 APB1 总线上
- A) I、II C) III、IV E) V、VI
B) II、III D) IV、V F) ABCDE 均不正确



三、 判断题 (14 分, 打√或×, 请试卷首页答题卡上答题)

- 30、ARM 处理器摒弃了冯·诺依曼结构, 均采用哈佛结构设计。
- 31、存储器芯片的地址线 and 数据线是不可以复用的。
- 32、计算机中缓存地址空间和主存地址空间是相同的地址空间。
- 33、AHB 和 APB 都具备独立的读写数据通道。
- 34、Cortex-M3 处理器流水线的 3 级是指取指令、指令译码、指令执行。
- 35、Cortex-M3 系统中, 子优先级高的异常可以打断子优先级低的异常。
- 36、Cortex-M3 系统中, MSP 的值不能设置为 0x0000 0000。
- 37、Cortex-M3 系统中, 异常的优先级寄存器只能由特权访问等级代码访问。
- 38、所有 Cortex-M3/M4 的存储器访问指令都支持非对齐的数据传输。
- 39、指令周期指的是 CPU 执行一条指令的时间。
- 40、Cortex-R 处理器利用多处理器锁步 (lock-step) 技术来提高可靠性。
- 41、STM32F407 系列处理器的每一根外部中断线都各自对应一个不同的中断向量。
- 42、微处理器最小硬件系统一般不包括 SPI、I²C 模块。
- 43、STM32F103 系列 MCU 只有 16 位的定时器, 故看门狗的最大时延是 65536 个 SYSCLK 周期。

四、 设计及综合题 (40 分, 每题 10 分, 6 题选做 4 题, 在稿纸上答题)

44、已知图 1 为某 Cortex-M3 系统中总线上数据传输的某个片段。综合运用课程知识, 并参考附录 3 信息, 回答下列问题。

- (1) 图 1 所示时序描述的 AHB 传输中数据的传输方向是什么?
- (2) HREADY 信号起什么作用?
- (3) 写出数据 B 和数据 C 对应的地址。
- (4) 若数据 B 的值为“0x2000 2000”、数据 C 的值为“0x0800 0800”, 分析数据 B、数据 C 各有何含义?
- (5) 依据 Cortex-M3 存储空间区域定义, 图 1 中总线操作访问的是哪个存储器区域? 并简要说明该区域的主要用途。

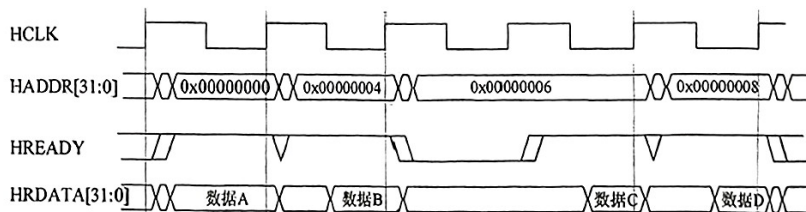


图 1 某 Cortex-M 系统 AHB 总线上的数据传输片段

45、某计算机系统采用 STM32F103ZE 处理器 (Cortex-M3, 32 根地址线, 32 根数据线)。现有数量充足的规格为 1MB×16 位的 SRAM 芯片, 试为该计算机设计一个具有 4MB 容量的片外扩展 SRAM 存储器子系统。(提示: Cortex-M3 定义地址范围 0x6000 0000~0x9FFF FFFF 是外部 RAM 区域)

- (1) 计算需要该型号 SRAM 芯片的数量。



(2) 画出地址线、数据线和 SRAM 芯片 CS 信号线的连接示意图（无需画出每根信号线和完整芯片，能展示信号连接关系即可）。

(3) 列出该 4MB 存储器的地址范围。

46、某计算机主存地址空间大小为 256 MB，按字节编址。指令 Cache 和数据 Cache 分离，均有 8 个 Cache 行，每个 Cache 行（cache line）大小为 64B，数据 Cache 采用直接相联映射方式。现有两个功能相同的程序 1 和程序 2，其伪代码如下所示。

程序 1	程序 2
<pre>int a[64][64]; int sum_array1() { int i, j, sum=0; for(i=0; i<64; i++) for(j=0; j<64; j++) sum += a[i][j]; return sum; }</pre>	<pre>int a[64][64]; int sum_array2() { int i, j, sum=0; for(j=0; j<64; j++) for(i=0; i<64; i++) sum += a[i][j]; return sum; }</pre>

假定 int 类型数据用 32 位补码表示；程序编译时 i, j, sum 分配在寄存器中；数组 a 按行优先方式存放，其首地址为 20D。回答下列问题，要求说明理由或给出计算过程。

- (1) 每个 Cache 行的 Tag 信息需要多少位？
- (2) 若不考虑用于 Cache 一致性维护和替换算法的控制位，则数据 Cache 的总容量（含 Tag 等信息）为多少？
- (3) 数组元素 a[0][31]和 a[2][1]各自所在的主存块对应的 Cache 行号分别是多少？（Cache 行号从 0 开始）
- (4) 定性说明哪个程序的执行时间更短？
- (5) 计算程序 1 的数据访问命中率（写出表达式即可）。

47、已知实验所用 STM32F407 实验箱上用拨动开关 DIP0 硬件连接如图 2 所示。拟设计的程序功能为：读取拨动开关 DIP0 状态（不接地时 PE4=1、接地时 PE4=0），通过 USART 串口模块发送所读取的状态值。部分代码如附录 2 所示。请结合课程知识、实验和附录 2 信息，回答以下问题。

- (1) 代码行①所定义的 GPIOE_BASE 的值是什么？
- (2) 代码行②所定义的 GPIOE 端口输入数据寄存器 IDR 的地址是什么？
- (3) 定性说明代码行③的功能。
- (4) 定性说明代码行④的功能。
- (5) 定性说明代码行⑤的功能。
- (6) 代码行⑥从哪个地址获取数据并赋值给变量 ui_dip0_value。
- (7) 代码行⑦和代码行⑧分别访问了 USART 模块的状态寄存器 SR 的 TXE 和 RXNE 标志位，解释这 2 个标志位的含义。

48、在实验所用 STM32F407 实验箱上用拨动开关 DIP0/DIP1 控制 LED0/LED1 亮灭。硬件连接如图 2 所示。拟设计的程序功能为：拨动开关 DIP0/DIP1 由不接地（PE4=1/PE5=1）拨动至接地（PE4=0/PE5=0）时，触发中断来实现 LED0 和 LED1 亮灭切换。部分代码如附录 1 所示。请结合课程知识，分析附录 1 代码，回答以下问题。



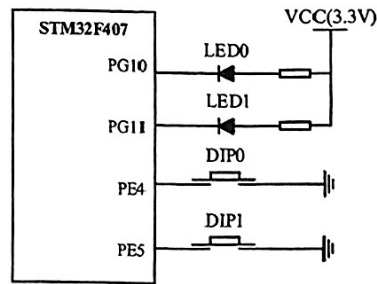


图 2 实验箱的 I/O 引脚连接示意图

- (1) 根据附录 3 表 1 关于 GPIO 寄存器的描述，解释代码行①和②的含义。
- (2) 简要说明 main() 函数所实现的功能。
- (3) 修改 EXTI9_5_IRQHandler() 函数代码，实现如下功能：在 DIP0 接地（PE4=0）时保持原有功能，而 DIP0 不接地时点亮 LED0 并熄灭 LED1。
- (4) 如果要修改附录 1 代码，实现如下功能：DIP0 拨动至接地（PE4=0）时，触发 EXTI4；在 EXTI4 中断服务子程序中熄灭 LED0 和 LED1；EXTI4 可以打断前述 EXTI5 中断服务。定性说明需要对附录 1 代码做哪些修改或补充。
- (5) 若希望程序执行更节能，可对 main() 函数做哪些修改或补充？定性说明即可。

49、图 3 所示为某 MCU 与外设的串行数据传输过程，SCLK 为同步时钟，SID 为数据。RW 为数据传送方向控制：H 表示数据从外设到 MCU，L 表示数据从 MCU 到外设。RS 为数据类型选择：H 表示数据是显示内容，L 表示数据是控制指令。D7~D4 为字节数据的高 4 位，D3~D0 为字节数据的低 4 位。传输过程中，只有在时钟线 SCLK 拉低时，数据线 SID 上的数据才允许变化，在时钟线 SCLK 高电平时，SID 上的数据必须保持稳定（不能变化）。

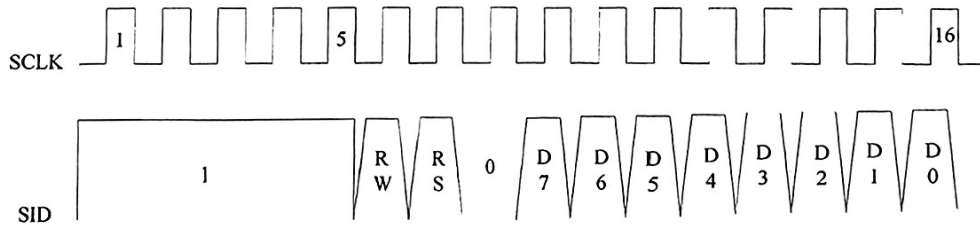


图 3 LCD12864 的串行数据传输过程

- (1) 根据图 3 所示数据传输示意图，发送一个字节数据，实际帧数据有多少字节？
- (2) 假设 SCLK 和 SID 已定义好对应引脚的位带(Bit-Band)操作，即代码“SCLK=1”会在 SCLK 信号线输出“1”，“SID=0”会在 SID 信号线输出“0”；可调用的延时函数为 delay_us(int N)。编写实现数据串行发送一个字节的函数 SendByte。

```
typedef unsigned char u8;
void SendByte(u8 byte)
{
    .....//补充代码
}
```
- (3) 已知该外设某条指令编码为 0x01，写出 MCU 发该指令操作的代码（调用函数 SendByte 实现）。



附录 1：中断相关代码（main.c）

```
//此处略去头文件信息
#define LED0_IsOn()    GPIO_ReadInputDataBit(GPIOG,GPIO_Pin_10)//读取 PG10
void LED0_Config (void);
void KEY0_Config(void);
void EXTI_Config (void);
void NVIC_Config (void);

int main (void)
{
    LED0_Config ();
    KEY0_Config ();
    EXTI_Config ();
    NVIC_Config ();
    while (1);
}

void EXTI_Config (void)
{
    EXTI_InitTypeDef EXTI_InitStructure;
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_SYSCFG, ENABLE);
    SYSCFG_EXTILineConfig(EXTI_PortSourceGPIOE, GPIO_PinSource5);
    EXTI_InitStructure.EXTI_Line= EXTI_Line5;
    EXTI_InitStructure.EXTI_Mode= EXTI_Mode_Interrupt;
    EXTI_InitStructure.EXTI_Trigger=EXTI_Trigger_Falling;
    EXTI_InitStructure.EXTI_LineCmd= ENABLE;
    EXTI_Init(&EXTI_InitStructure);
}

void NVIC_Config (void)
{
    NVIC_InitTypeDef NVIC_InitStructure;
    NVIC_PriorityGroupConfig (NVIC_PriorityGroup_1);
    NVIC_InitStructure.NVIC_IRQChannel= EXTI9_5_IRQn;
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority= 0;
    NVIC_InitStructure.NVIC_IRQChannelSubPriority= 1;
    NVIC_InitStructure.NVIC_IRQChannelCmd= ENABLE;
    NVIC_Init (&NVIC_InitStructure);
}

void LED0_Config (void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOE | RCC_AHB1Periph_GPIOG,
ENABLE);
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_11 | GPIO_Pin_10;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_OUT;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
    GPIO_Init(GPIOG, &GPIO_InitStructure);
}
```



```

void KEY0_Config (void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOE, ENABLE);
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_4 | GPIO_Pin_5;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_100MHz;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_UP;
    GPIO_Init (GPIOE, &GPIO_InitStructure);
}

void EXTI9_5_IRQHandler (void)
{
    if (EXTI_GetITStatus (EXTI_Line5) != RESET)
    {
        if (LED0_IsOn())
            GPIOG->BSRR = 0x0C000000 //①
        else
            GPIOG->BSRR = 0x00000C00; //②
        EXTI_ClearITPendingBit (EXTI_Line5);
    }
}

void EXTI4_IRQHandler (void)
{
}

```

附录 2: USART 相关代码 (stm32f4xx.h、main.c)

ST 官方库文件 stm32f4xx.h 代码片段

```

#define PERIPH_BASE ((uint32_t)0x40000000)
#define AHB1PERIPH_BASE (PERIPH_BASE + 0x00020000)
#define GPIOE_BASE (AHB1PERIPH_BASE + 0x1000) //①
#define GPIOE ((GPIO_TypeDef *) GPIOE_BASE)

typedef struct
{
    __IO uint32_t MODER; /*Address offset: 0x00 */
    __IO uint32_t OTYPER; /*!Address offset: 0x04 */
    __IO uint32_t OSPEEDR; /*!Address offset: 0x08 */
    __IO uint32_t PUPDR; /*!Address offset: 0x0C */
    __IO uint32_t IDR; /*!Address offset: 0x10 */
    __IO uint32_t ODR; /*!Address offset: 0x14 */
    __IO uint16_t BSRRL; /*!Address offset: 0x18 */
    __IO uint16_t BSRRH; /*!Address offset: 0x1A */
    __IO uint32_t LCKR; /*!Address offset: 0x1C */
    __IO uint32_t AFR[2]; /*!Address offset: 0x20-0x24 */
} GPIO_TypeDef;

```




```

main.c
#include "stm32f10x.h"

#define BITBAND(addr, bitnum) ((addr & 0xF0000000)+0x20000000+((addr & 0xFFFF)<<5)+(bitnum<<2))
#define MEM_ADDR(addr) *((volatile unsigned long *) (addr))
#define BIT_ADDR(addr, bitnum) MEM_ADDR(BITBAND(addr, bitnum))
#define GPIOE_IDR_Addr (GPIOE_BASE+16) //②
#define GPIOE_ODR_Addr (GPIOE_BASE+20)
#define PEin(n) BIT_ADDR(GPIOE_IDR_Addr,n) //③
#define PEout(n) BIT_ADDR(GPIOE_ODR_Addr,n)

void GPIOD_Config(void)
{
    GPIO_InitTypeDef GPIO_InitStructure;
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOD,ENABLE); //④
    GPIO_PinAFConfig(GPIOD,GPIO_PinSource5,GPIO_AF_USART2);
    GPIO_PinAFConfig(GPIOD,GPIO_PinSource6,GPIO_AF_USART2); //⑤
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_5 | GPIO_Pin_6;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_InitStructure.GPIO_OType = GPIO_OType_PP;
    GPIO_InitStructure.GPIO_PuPd = GPIO_PuPd_NOPULL;
    GPIO_Init(GPIOD,&GPIO_InitStructure);
}

int main (void)
{
    unsigned int ui_dip0_value = 0;
    unsigned char uc_send_ch = 0;
    unsigned char uc_recv_ch = 0;
    GPIOD_Config();
    //GPIOE 初始化代码，此处略去
    //USART2 初始化代码，此处略去
    while (1) {
        ui_dip0_value = PEin(4); //⑥
        uc_send_ch = ui_dip0_value; //unsigned int 类型转换为 unsigned char
        if(USART_GetFlagStatus(USART1,USART_FLAG_TXE)==SET) //⑦
            USART_SendData (USART2, uc_send_ch); //向 USART2 发送数据
        if (USART_GetFlagStatus (USART2, USART_FLAG_RXNE) !=RESET) //⑧
            uc_recv_ch = USART_ReceiveData(USART2); //从 USART2 接收数据
    }
}

```



附录 3：STM32F407 系列 MCU 相关信息

表 1 STM32F407 的 GPIO_BSRR 寄存器位定义

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
BR15	BR14	BR13	BR12	BR11	BR10	BR9	BR8	BR7	BR6	BR5	BR4	BR3	BR2	BR1	BR0
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
BS15	BS14	BS13	BS12	BS11	BS10	BS9	BS8	BS7	BS6	BS5	BS4	BS3	BS2	BS1	BS0

GPIO_BSRR[31:16] BRy: Port x reset bit y (y=0..15)
BRy 为 1, 表示 Port x 的输出数据寄存器 (ODRx) 的 y 位置 0, 即 Port x 的 y 位输出 0
GPIO_BSRR[15:0] BSy: Port x set bit y (y=0..15)
BSy 为 1, 表示 Port x 的输出数据寄存器 (ODRx) 的 y 位置 1, 即 Port x 的 y 位输出 1

表 2 STM32F407 系列 MCU 的中断向量表 (部分)

Position	Priority	Type of priority	Acronym	Description	Address
-	-	-	-	Reserved	0x0000 0000
-	-3	fixed	Reset	Reset	0x0000 0004
-	6	settable	SysTick	System tick timer	0x0000 003C
0	7	settable	WWDG	Window Watchdog interrupt	0x0000 0040
10	17	settable	EXTI4	EXTI Line0 interrupt	0x0000 0068
23	30	settable	EXTI9_5	EXTI Line[9:5] interrupts	0x0000 009C
28	35	settable	TIM2	TIM2 global interrupt	0x0000 00B0
37	44	settable	USART1	USART1 global interrupt	0x0000 00D4
38	45	settable	USART2	USART2 global interrupt	0x0000 00D8
39	46	settable	USART3	USART3 global interrupt	0x0000 00DC

附录 4：ARM 汇编指令相关信息

ADD 指令

加法指令, 典型语法是 “ADD Rd, Rn, Rm”, 该指令将 Rn+Rm 的结果存入 Rd。

LDR 指令

从存储器读取字数据至寄存器。典型语法为 “LDR Rd, [Rn, #offset]”, 该指令从存储器位置 Rn+offset 读取一个字存入寄存器 Rd。若指定操作数选址方式为前变址寻址 (从位置 Rn+offset 读取一个字, 更新 Rn 为 Rn+offset), 其典型语法为 “LDR Rd, [Rn, #offset]!”。若指定操作数选址方式为后变址寻址 (从位置 Rn 读取一个字, 更新 Rn 为 Rn+offset), 其典型语法为 “LDR Rd, [Rn], #offset”。

附录 5：数的表示方法

补码

数值 x 的补码记为 $[x]_n$, 假设机器字长为 n 位, 那么数值 x 的补码定义如下:

$$[x]_n = \begin{cases} x & 0 \leq x \leq 2^{n-1} - 1 \\ 2^n - |x| & -2^{n-1} \leq x \leq 0 \end{cases}$$

浮点数

在国际电气和电子工程师协会 IEEE-754 标准中, 32 位浮点数的格式如下:

	31	30		23	22		0
32位浮点数	S		e		M		

S 位是 1 bit 符号位, e 是 8 bit 阶码 (隐含基数为 2), M 是 23 bit 的尾数。IEEE-754 规定, 上述格式所表示浮点数的真值为 $(-1)^S \times 2^{e-127} \times (1 + M)$ 。

